

ComicCraft

AI Comic Story Creator using Gemini Models

Full Project Documentation

Project Report, System Design, Implementation, Testing, Setup and Demonstration

Project	ComicCraft - AI Comic Story Creator using Gemini Models
Team Leader	Boopathi P.
Team Members	Ganesh S.; Muthu Sudar A.; Pradeep Durai D.; Akshay Akash S.
Frontend	React + Vite
Backend	Python + FastAPI + Uvicorn
AI Platform	Google Gemini Models
Project Type	Educational / Academic Project

Prepared as a complete academic project document

Table of Contents

- 1. Project Overview
- 2. Abstract
- 3. Problem Statement
- 4. Existing vs Proposed System
- 5. Objectives
- 6. Scope
- 7. Key Features
- 8. Functional Requirements
- 9. Non-Functional Requirements
- 10. Technology Stack
- 11. System Architecture
- 12. Workflow
- 13. Modules
- 14. Data Flow and API Design
- 15. Frontend Design
- 16. Backend Design
- 17. Gemini Integration
- 18. Project Structure
- 19. Installation and Run Commands
- 20. Testing and Test Cases
- 21. Error Handling and Limitations
- 22. Security
- 23. Team Responsibilities
- 24. Future Enhancements
- 25. Benefits and Expected Outcome
- 26. Demonstration Flow
- 27. Conclusion

1. Project Overview

ComicCraft is a web-based AI comic story creation application that converts a user's creative idea into a structured comic storyboard. The user provides a story idea, selects a genre and chooses the desired panel count. The backend sends the request to Google Gemini models and returns structured story, character and panel information.

The project combines a React + Vite frontend with a Python + FastAPI backend. Image generation is implemented as an additional capability for accounts/models that have access to Gemini image generation.

2. Abstract

Comic creation normally requires plot development, character design, scene planning, dialogue writing and panel organization. ComicCraft uses Generative AI to automate these preparation steps. A short prompt can be transformed into a complete story concept with characters, scenes and dialogue.

The application demonstrates a practical full-stack AI architecture: React provides the user interface, FastAPI handles REST requests and validation, Python manages server-side Gemini integration, and Gemini provides generative intelligence. Secret credentials remain on the backend rather than being exposed in browser code.

3. Problem Statement

Creating a comic manually can be time-consuming, especially for beginners. A creator must turn an idea into a coherent story, define characters, write dialogue, divide events into panels and plan visuals. ComicCraft addresses the problem of converting a simple natural-language idea into a structured, readable and visually organized comic storyboard using Generative AI.

4. Existing vs Proposed System

Aspect	Traditional / Manual	ComicCraft
Story	Written manually	AI-assisted generation
Characters	Designed manually	Generated from story context
Panels	Manually planned	AI-generated storyboard structure
Dialogue	Written manually	Generated with the story
Workflow	Multiple tools	Single web application
Iteration	Manual rewriting	New generation from revised prompt

5. Objectives

- Build a user-friendly AI comic creation interface.
- Generate a structured story from a short prompt.
- Automatically generate character details and comic panels.
- Integrate Google Gemini through a secure backend.
- Display results as an interactive storyboard.
- Support AI-generated panel images when available.
- Demonstrate a complete frontend-backend-AI workflow.

6. Scope

The project covers story input, genre selection, panel count selection, AI story generation, character generation, storyboard generation, frontend presentation, backend API integration and optional image generation. Authentication, cloud storage, collaborative editing and advanced production-grade visual consistency are outside the current prototype and can be future enhancements.

7. Key Features

- Story idea input
- Genre selection
- Panel count selection
- AI story generation
- Automatic character generation
- Comic storyboard with scenes and dialogue
- Gemini image-generation support
- Modern dark-purple React interface
- FastAPI Swagger documentation
- Secure backend API-key configuration

8. Functional Requirements

ID	Requirement	Description
FR-01	Story Input	User can enter a natural-language comic idea.
FR-02	Genre	User can select a genre.
FR-03	Panel Count	User can select desired number of panels.
FR-04	Story Generation	System generates structured story content.
FR-05	Characters	System displays generated character information.
FR-06	Storyboard	System displays titles, scenes and dialogue.
FR-07	Images	System can request panel images when access is available.
FR-08	Error Handling	System reports validation, API and quota failures clearly.

9. Non-Functional Requirements

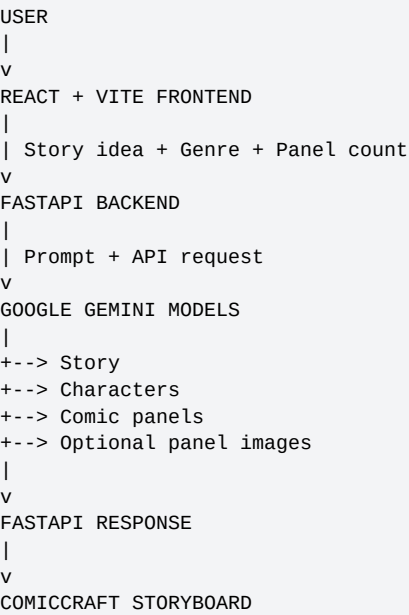
- Usability: simple interface for first-time users.
- Performance: asynchronous requests keep the UI responsive.
- Security: Gemini API key remains on the backend.
- Maintainability: frontend, backend and AI logic are separated.
- Reliability: external API failures should not crash the complete application.
- Scalability: additional models and export features can be added later.

10. Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite	Interactive web interface
Styling	CSS	Visual design and responsive layout
Backend	Python + FastAPI	REST API and server logic
Server	Uvicorn	Runs FastAPI
Configuration	python-dotenv	Loads .env variables
AI SDK	Google GenAI SDK	Gemini API integration
API Testing	FastAPI Swagger	Interactive endpoint testing

11. System Architecture

ComicCraft uses a client-server architecture with Gemini as the AI service layer. The browser communicates with FastAPI, and FastAPI communicates with Gemini. This keeps the API key out of frontend JavaScript.



12. Application Workflow

- Open ComicCraft.
- Choose Create.
- Enter a story idea.
- Select genre and panel count.
- React sends POST /generate-story.
- FastAPI validates the request and calls Gemini.
- Gemini returns structured story data.
- React displays title, summary, characters and panels.
- Optional image requests are sent to /generate-panel-image.
- Final comic storyboard is displayed.

13. Module Description

Module	Responsibility
Home	Project introduction and navigation
Create Form	Collects prompt, genre and panel count
Story Generator	Calls story-generation API
Character Section	Displays names, roles and personalities
Storyboard	Displays panel title, scene and dialogue
Image Generator	Requests visual panel images when available
Backend API	Validates requests and manages Gemini calls
Configuration	Stores API credentials securely

14. Data Flow and API Design

The main story request is JSON based. The frontend sends prompt, genre and panel_count to the backend.

```
POST /generate-story

{
  "prompt": "A student discovers a magical tree on campus",
  "genre": "Fantasy",
  "panel_count": 6
}
```

An image request contains the information required to illustrate a single panel.

```
POST /generate-panel-image

{
  "panel_number": 1,
  "panel_title": "Into the Unknown",
  "scene": "The hero enters a mysterious forest...",
  "dialogue": "I have never seen this place before.",
  "characters": [],
  "genre": "Fantasy"
}
```

The backend returns structured JSON. When an image is available, the response includes image data and its MIME type so the browser can display it.

15. Frontend Design

The frontend uses React and Vite. The application manages Home, Create, Generating and Result states. The visual design uses a dark-purple comic-inspired theme with navigation, cards, buttons, loading indicators and storyboard panels.

- Home: project introduction.
- Create: story, genre and panel inputs.
- Generating: AI processing state.
- Result: story, characters and storyboard.

16. Backend Design

FastAPI provides REST endpoints, automatic request validation and Swagger documentation. The backend loads GEMINI_API_KEY from environment variables, creates a Gemini client, constructs prompts and returns structured results. CORS is configured so the local React development server can call the backend.

17. Gemini AI Integration

Gemini is the generative intelligence layer. Text generation creates the story, characters and storyboard. Image generation can convert individual panel descriptions into comic artwork when the selected image model and account have the necessary access and quota.

Image generation is therefore treated as an optional capability. If the account has no image-generation quota, the story and storyboard can still be displayed. Model availability, quotas and pricing are external service conditions and should be checked against Google's current documentation before deployment.

18. Project Structure

```
comiccraft-ai-comic-story-creator/
├── README.md
├── 01-Brainstorming-and-Ideation/
├── 02-Requirement-Analysis/
├── 03-Project-Design/
├── 04-Project-Planning/
├── 05-Project-Development/
├── Frontend/
└── Backend/
```

```

■ ■■■ Gemini-Integration/
■ ■■■ Screenshots/
■■■ 06-Project-Testing/
■■■ 07-Project-Documentation/
■■■ 08-Project-Demonstration/
■■■ assets/
■■■ frontend/
■■■ backend/
■■■ main.py
■■■ .env
■■■ requirements.txt
■■■ venv/

```

19. Installation and Run Commands

19.1 Frontend Setup

```

cd frontend
npm install
npm run dev

```

Open the Vite URL shown in the terminal. The current development setup uses **http://localhost:5174**.

19.2 Backend Setup

```

cd backend
python -m venv venv
venv\Scripts\activate
python -m pip install -U google-genai fastapi uvicorn python-dotenv

```

19.3 Gemini API Key

```
GEMINI_API_KEY=YOUR_REAL_API_KEY
```

Save this in **backend/.env**. Never put the key inside frontend source code.

19.4 Start Backend

```

cd backend
venv\Scripts\activate
python -m uvicorn main:app --reload --port 8000

```

Backend: **http://localhost:8000**

Swagger: **http://localhost:8000/docs**

19.5 Complete Run

```

# Terminal 1
cd backend
venv\Scripts\activate
python -m uvicorn main:app --reload --port 8000

# Terminal 2
cd frontend
npm run dev

# Browser
http://localhost:5174

```

20. Testing and Test Cases

ID	Test Case	Expected Result
TC-01	Open Home	Home page loads successfully
TC-02	Empty story input	Validation prevents invalid request
TC-03	Valid story generation	Story, characters and panels are returned
TC-04	Character display	Character cards are visible

ID	Test Case	Expected Result
TC-05	Storyboard display	Panel title, scene and dialogue are visible
TC-06	Image generation with access	Generated image is displayed
TC-07	Image quota unavailable	Storyboard remains usable and image failure is handled
TC-08	Backend unavailable	Frontend shows a useful connection error
TC-09	Swagger	/docs opens successfully
TC-10	Secret protection	API key is not exposed in frontend

21. Error Handling and Limitations

Condition	Meaning	Handling
400	Invalid request/model parameter	Show configuration or request error
422	FastAPI validation error	Check request fields and values
429	Rate/quota limit	Avoid unnecessary retries and explain availability
500	Backend exception	Show user-friendly error and log details
Network failure	Frontend cannot reach backend	Check backend and port 8000

A key project limitation is that Gemini image generation may not be available on every account or plan. The core story and storyboard should therefore remain usable even when image generation is unavailable.

22. Security Considerations

- Keep the Gemini API key only in backend/.env.
- Add .env and venv/ to .gitignore.
- Never hard-code the secret in React.
- Do not print the key in browser logs.
- Validate API input.
- Restrict CORS to trusted origins in production.
- Use HTTPS for production.

```
# .gitignore
venv/
.env
__pycache__/
node_modules/
dist/
```

23. Team Responsibilities

Member	Suggested Responsibility
Boopathi P.	Architecture, integration, coordination and final project assembly
Ganesh S.	Frontend/UI development
Muthu Sudar A.	Backend/API development
Pradeep Durai D.	Gemini AI integration and prompting
Akshay Akash S.	Testing, documentation and demonstration

The responsibility allocation is a practical project-organization proposal and can be adjusted to match the team's actual contributions.

24. Future Enhancements

- User accounts and project history
- Save and reopen comic projects
- Editable stories, characters and panels
- Character reference images for consistency
- Multiple art styles and panel layouts
- PDF/PNG comic export
- Downloadable complete comic book
- Voice-to-story input
- Multi-language generation
- Cloud deployment and database
- Advanced moderation and content-safety controls

25. Benefits and Expected Outcome

- Reduces comic creation time.
- Helps beginners create structured comic stories.
- Automates story and character generation.
- Provides an interactive AI-powered experience.
- Demonstrates practical Generative AI integration.
- Creates a foundation for future AI-assisted visual storytelling.

26. Demonstration Flow

1. Introduce the problem and project objective.
2. Show the ComicCraft home page.
3. Open Create screen.
4. Enter a sample story idea.
5. Select genre and panel count.
6. Click Generate Story.
7. Show generated story and characters.
8. Show comic storyboard panels.
9. Demonstrate image generation if available.
10. Open FastAPI /docs and explain the API.
11. Explain architecture and Gemini integration.
12. Present benefits, limitations and future scope.

27. Conclusion

ComicCraft demonstrates how Generative AI can be combined with a modern web stack to create an interactive comic-story creation platform. The system converts a simple idea into structured story content containing characters, scenes, dialogue and storyboard panels.

The project also demonstrates full-stack engineering practices including frontend-backend separation, REST API design, environment-based secret management, asynchronous processing and external-service error handling. Gemini provides the generative intelligence while React and FastAPI provide the application framework.

With project persistence, editing, consistent character imagery and export features, ComicCraft can be extended into a more complete AI-assisted storytelling platform.

Appendix A - Quick Start

```
# Backend
cd backend
venv\Scripts\activate
python -m uvicorn main:app --reload --port 8000

# Frontend
cd frontend
npm install
npm run dev

# Open
http://localhost:5174

# Swagger
http://localhost:8000/docs
```

Appendix B - Project Summary

Item	Details
Project Title	ComicCraft - AI Comic Story Creator using Gemini Models
Domain	Generative AI / Web Application
Frontend	React + Vite
Backend	Python + FastAPI
AI	Google Gemini Models
Primary Output	AI-generated story and comic storyboard
Optional Output	AI-generated comic panel images
Project Type	Educational / College Project